

Akamata

A modern Zig 0.16 web framework

One source — runs on a VPS, runs on Cloudflare Workers

v0.3 · 2026

Why we built it

- Bring **Zig**'s performance + safety (zero deps, explicit memory) to the web
- Ship "**a small static binary for VPS**" AND "**a Cloudflare Workers wasm**" from **one codebase**
- Hono-style DX (handlers, middleware, typed JSON) in Zig
- Transparent DB layer: SQLite / Turso / D1 swappable by URL

Result

Production-quality · 53M requests with zero leak · P99 88μs · audited HTTP smuggling defense

Hello, Akamata

If you've used Express or Hono, you already know how to read this.

```
const std = @import("std");
const am  = @import("akamata");

const State = struct {};

pub fn main() !void {
    var gpa: std.heap.DebugAllocator(.{}) = .init;
    defer _ = gpa.deinit();

    var app = am.App(State).init(gpa.allocator(), .{});
    defer app.deinit();

    _ = try app.useAll(am.mw.recover(State));
    _ = try app.get("/", hello);

    try app.serve(.{ .port = 8080 });
}
```

One source, two deploy shapes

Native binary

```
zig build -Doptimize=ReleaseFast
# static musl binary
zig build \
  -Dtarget=x86_64-linux-musl \
  -Doptimize=ReleaseFast
```

VPS · Docker · Cloudflare Containers

Cloudflare Workers

```
zig build \
  -Dbackend=workers \
  -Doptimize=ReleaseSmall
# ~140 KB wasm
akamata deploy --workers
```

Edge · D1 / Turso · scales

Handler code is **identical**. Use the `am.backend` constant only where you really need to branch.

Swap the DB with a URL

```
var db = try am.db.open(alloc, url);
```

URL scheme	Backend	Runs where
<code>file:./app.db</code>	SQLite (in-process)	native only
<code>libsql://<db>.turso.io?authToken=...</code>	Turso (libsql HTTP)	native + Workers
<code>https://<db>.turso.io?authToken=...</code>	Turso (HTTPS alias)	native + Workers
<code>d1:DB</code>	Cloudflare D1 binding	Workers only

Handlers see the same `am.db.Db` vtable. **Same SQL, same bind args, no branching.**

The Model layer — one struct does it all

```
pub const Todo = struct {
    id: ?i64 = null,
    title: []const u8,
    priority: i32 = 3,
    completed: bool = false,
    created_at: ?i64 = null,

    pub const __schema = .{
        .table = "todos",
        .indexes = .{ .{ .{ "completed", "priority" }, .index } },
        .defaults = .{ .created_at = "unixepoch()" },
        .validates = .{
            .title = .{ am.model.rule.required, am.model.rule.max_len(200) },
            .priority = .{ am.model.rule.range(1, 3) },
        },
    };
};
```

Validation, defaults, indexes, DDL, migrations — all derived from this one declaration

Typed CRUD — no raw SQL needed

```
// reads
const one    = try Todos.find(db, arena, 42);           // ?Todo
const all    = try Todos.all(db, arena);               // []Todo
const highs  = try Todos.where(db, arena, .{ .priority = 1 });

// writes
const created = try Todos.create(db, arena, .{ .title = "buy milk" });
try Todos.save(db, arena, &updated);
try Todos.delete(db, 7);

// escape hatch – arbitrary SQL still maps to T
const rows = try Todos.queryRaw(db, arena,
    "SELECT * FROM todos WHERE created_at > ? ORDER BY id", .{cutoff});

// kills N+1 – one IN query fetches all children
const loaded = try am.model.preload.hasMany(User, "posts", owners, db, arena);
```

Handlers, abbreviated

Before — 13 lines

```
fn createTodo(c: *Ctx) !void {
  const Body = struct { title: []const u8 };
  const body = c.req.json(Body) catch {
    return c.json(.{
      .error_kind = "bad_request",
      .message = "expected {title}",
    }, 400);
  };
  const errs = try Todos.validate(body, c.arena);
  if (errs.len > 0) {
    return c.json(.{ .errors = errs }, 422);
  }
  const t = try Todos.create(c.db(), c.arena, body);
  try c.json(t, 201);
}
```

After — 3 lines

```
fn createTodo(c: *Ctx) !void {
  const todo = (try c.input(Todo)) orelse return;
  const t = try Todos.create(c.db(), c.arena, todo);
  try c.json(t, 201);
}
```

- `c.input(T)` parses + validates
- On failure **400 / 422 auto-emitted**
- `orelse return` for early exit

Auto-migration

At startup, diff the model's `__schema` against the live DB, generate only the needed ALTERs.

```
const plan = try am.model.migrate.diff(arena, db, &all_models);  
try am.model.migrate.apply(arena, db, plan);  
// → CREATE TABLE / ADD COLUMN / CREATE INDEX as needed
```

Or versioned files

```
$ akamata migrate generate add_email  
→ creates migrations/20260523120000_add_email.sql  
  
$ ./myapp migrate-up  
→ checks schema_migrations, applies pending ones
```

Both modes work on SQLite / Turso / D1 alike. On Workers it goes through JSPI.

One command deploy

```
$ akamata deploy --workers \  
  --config=deploy/wrangler.toml \  
  --migrate=<(. /zig-out/bin/myapp --print-schema)
```

1. Reads `[[d1_databases]]` from `wrangler.toml`
2. Placeholder UUID? Runs `wrangler d1 create`, writes the real one back
3. DB already exists? Picks it up via `wrangler d1 list`
4. Applies the schema you piped through `--migrate`
5. `zig build -Dbackend=workers`
6. `wrangler deploy`

The magic: JSPI

Lets Zig call D1's async API **as if it were synchronous**.

JS host (index.mjs)

```
d1_step: new WebAssembly.Suspending(
  async (h) => {
    const e = d1stmts.get(h);
    return await e.iter.next();
  }
);

handleFetchAsync = WebAssembly.promising(
  exports.handle_fetch);
```

Zig side (d1.zig)

```
extern "akamata_d1"
    fn d1_step(stmt: i32) i32;

// looks synchronous
while ((try stmt.step()) == .row)
    try rows.append(arena, row);
```

Reliability — production audited

HTTP smuggling defense

CL + TE rejected, duplicate CL rejected, malformed chunked rejected. **902 adversarial fuzz attempts, zero panics.**

JWT done right

alg=none / alg=RS256 spoof rejected, HS256 explicitly verified (CVE-2015-2951), time validation mandatory.

Stable under load

53M requests in 5 minutes, RSS flat (no leak), P99 didn't drift.

TLS cert verification

SAN/CN check, SSL_VERIFY_PEER forced. MITM-safe default.

Graceful shutdown

SIGINT/SIGTERM closes the listener, drains. Zero-downtime restarts.

OOM safe

Per-connection arena isolation, errors carried in error unions. Every panic site audited.

Performance

vs. Go / Hono

Scenario	Akamata	Go	Hono+Bun
/hello (text)	175k	208k	165k
/echo (JSON)	181k	175k	128k
/db (SQLite)	91k	46k (40% err)	138k

req/s, M2 Pro 8 workers · 5 minutes sustained

88 μ s

P99 latency (5-min sustained)

2.5 MB

RSS after 53M requests

Observability out of the box

```
$ curl https://app.example.com/metrics
```

```
akamata_requests_total 5891
akamata_requests_by_status{class="2xx"} 5700
akamata_requests_by_status{class="5xx"} 3
akamata_requests_by_method{method="POST"} 1280
akamata_request_latency_seconds_bucket{le="0.0001"} 4810
akamata_request_latency_seconds_sum 0.872413
akamata_process_resident_memory_bytes 2621440
akamata_process_uptime_seconds 1234
```

- Fixed cardinality keeps your TSDB happy (no path labels, methods are 8 fixed values)
- Structured access logs (JSON, request_id auto-added)
- Just point Prometheus at it with `scrape_interval: 15s`

akamata CLI

```
# scaffold (both native + Workers)
$ akamata init myapp --target=both

# dev
$ akamata dev                # alias for zig build run

# migrations
$ akamata migrate generate add_users
$ akamata migrate up

# deploy
$ akamata deploy --workers --migrate=<(/myapp --print-schema)
$ akamata deploy --containers

# typo-friendly
$ akamata deplyo
akamata: unknown subcommand `deplyo`
Did you mean `akamata deploy`?
```

Batteries included

recover

Traps panics, replies 500. Wrap as the outermost layer.

logger / accessLog

stdout / structured JSON access log

requestId

X-Request-ID generated and propagated

metrics

Prometheus text exposition

jwt / bearerAuth

HS256/RS256, CVE-2015-2951-safe

cors

Preflight + Origin allowlist

rateLimit

Token bucket per-IP / per-key

session / csrf

Signed cookies + double-submit

serveStatic

File serving with ETag, mime sniffing

WebSocket — upgrade from any route

```
_ = try app.ws("/rooms/:id/ws", wsRoom);

fn wsRoom(c: *Ctx) !void {
  const room_id = try c.req.paramAs(u64, "id");
  var conn = try am.ws.upgrade(Ctx, c, .{ .max_message_bytes = 64 * 1024 });
  try c.state().hub.attach(room_id, &conn);
  defer c.state().hub.detach(room_id, &conn);

  while (true) {
    const msg = conn.readMessage() catch return;
    defer msg.deinit();
    if (msg.opcode == .text)
      try c.state().hub.broadcast(room_id, msg.payload);
  }
}
```

On Workers, WS sessions are delegated to a Durable Object automatically — your handler code doesn't change.

Validation — declare it, done

```
pub const __schema = .{
    .validates = .{
        .email = .{
            am.model.rule.required,
            am.model.rule.max_len(255),
            am.model.rule.format(.email),
        },
        .age = .{ am.model.rule.range(0, 150) },
        .name = .{ am.model.rule.required, am.model.rule.custom(noScriptTag) },
    },
};

fn noScriptTag(value: []const u8, _: std.mem.Allocator) ?[]const u8 {
    if (std.mem.indexOf(u8, value, "<script>") != null)
        return "must not contain <script>";
    return null;
}
```

Built-ins: `required`, `min_len`, `max_len`, `range`, `format(.email/.url/.alphanumeric)`, `custom`, `customInt`

Eager loading — defeat N+1

```
// User has_many :posts
const users = try Users.all(db, arena);           // 1 query
const loaded = try am.model.preload.hasMany(
    User, "posts", users, db, arena);             // 1 IN query

for (loaded) |row| {
    // row.parent: User
    // row.related: []Post ← already loaded
    std.log.info("{s} has {d} posts", .{
        row.parent.name, row.related.len
    });
}
```

N parents → **2 queries total**. Children fetched in batch via `WHERE user_id IN (?, ?, ?, ...)`.

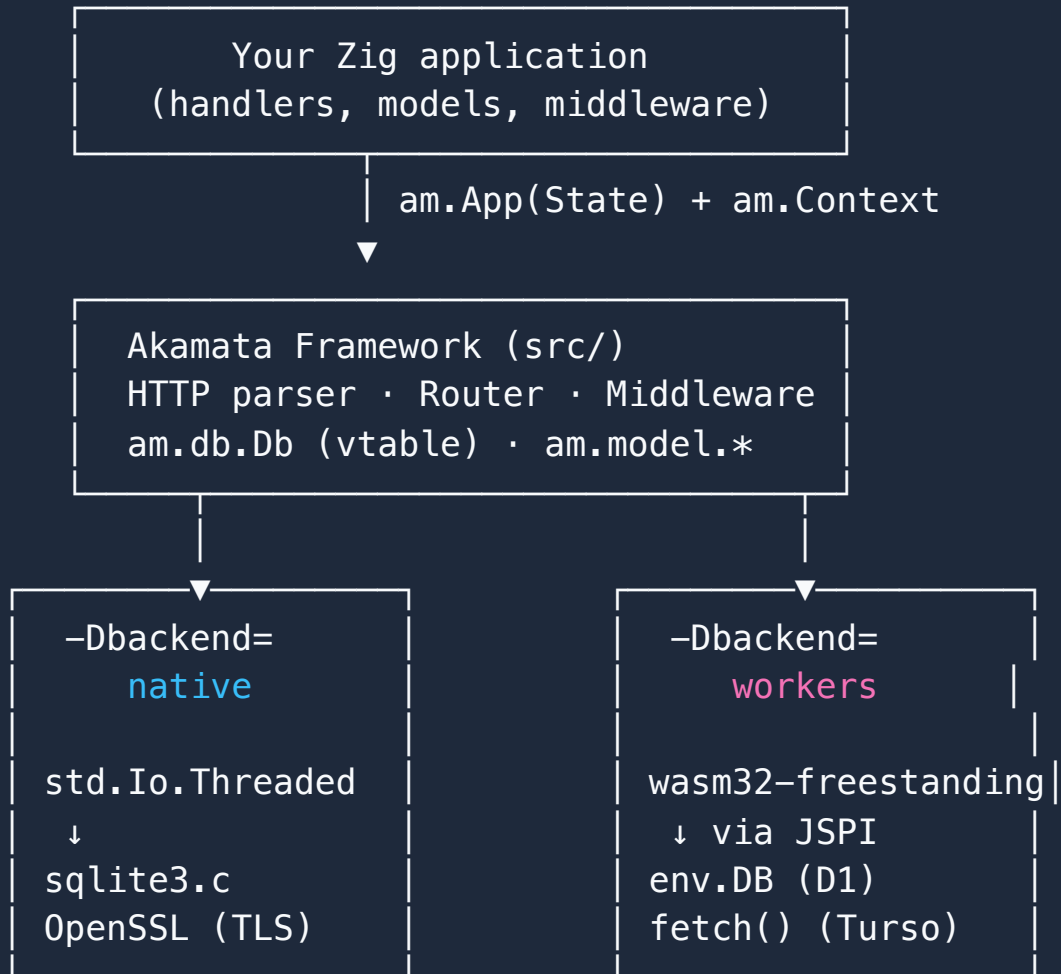
Ship the HTML UI in the binary

```
const index_html = @embedFile("index.html");

fn index(c: *Ctx) !void {
    const accept = c.req.header("accept") orelse "";
    if (std.mem.indexOf(u8, accept, "text/html") != null) {
        return c.html(index_html); // browsers → HTML UI
    }
    try c.json(.{ .endpoints = .{ ... } }, 200); // curl → JSON
}
```

- `@embedFile` bakes the file into the binary/wasm at compile time
- Single artifact serves everything — no extra deploy step
- Works **without a separate CDN** on Workers too

The shape of it



Who's it for

Serverless APIs

Workers + D1, scales to edge. Cold start ~30ms, wasm ~150KB.

Mobile / SaaS backends

JWT, friends, messaging, push — all batteries included.

Realtime

WS hub, Durable Object integration, room/user broadcast.

Microservices

1.8 MB static binary, ~10ms start, metrics built in.

Security-first

Production-audited: HTTP smuggling, JWT spoof, MITM defense.

CRUD / admin UIs

Model + auto-migrate + validation + HTML embed = one file.

How it compares

	Akamata	Hono (Bun/Node)	Go net/http	Express
Language	Zig	TypeScript	Go	JavaScript
VPS deploy	✓ static musl	⚠ runtime needed	✓ static	⚠ Node needed
Workers	✓ wasm	✓ native	✗	✗
Sync D1 calls	✓ JSPI	N/A (async)	—	—
SQLite bundled	✓	via bun:sqlite	external	external
P99 (echo)	87 µs	~250 µs	~90 µs	~500 µs
Binary size (native)	1.8 MB	~80 MB (bun)	~10 MB	node + npm

Roadmap

✓ v0.3 (now)

- Model layer + Repo + auto-migration
- Eager loading / relations / validation
- Full D1 via JSPI
- One-shot akamata deploy
- Production audit (reliability / observability)
- Detailed tutorial + PDF

🚧 Planned

- OpenTelemetry export
- Reactor + io_uring / kqueue for more throughput
- Enum ↔ TEXT column auto-mapping
- GraphQL integration (experimental)
- Hot reload (dev mode)
- WebTransport / HTTP/3

Get started

```
git clone https://github.com/yourorg/Akamata
cd Akamata && zig build cli
akamata init myapp --target=both
cd myapp && zig build run
```

 **Detailed tutorial:** `docs/tutorial.md` (60–90 min, builds a Todo app)

 Handbook: `docs/handbook.md` (15-min tour)

 Samples: `examples/{guestbook,chat,mobus}`

Happy building with Akamata 